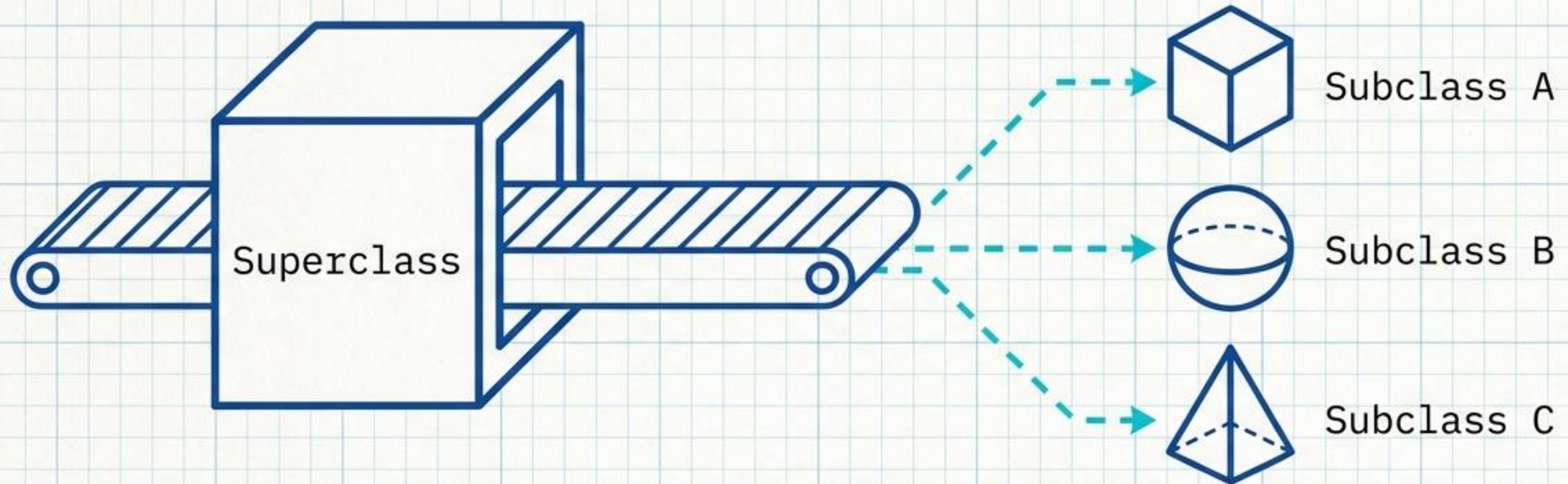


Factory Method Pattern

Prepared By: Team 2

Instructor: Md Samsuddoha



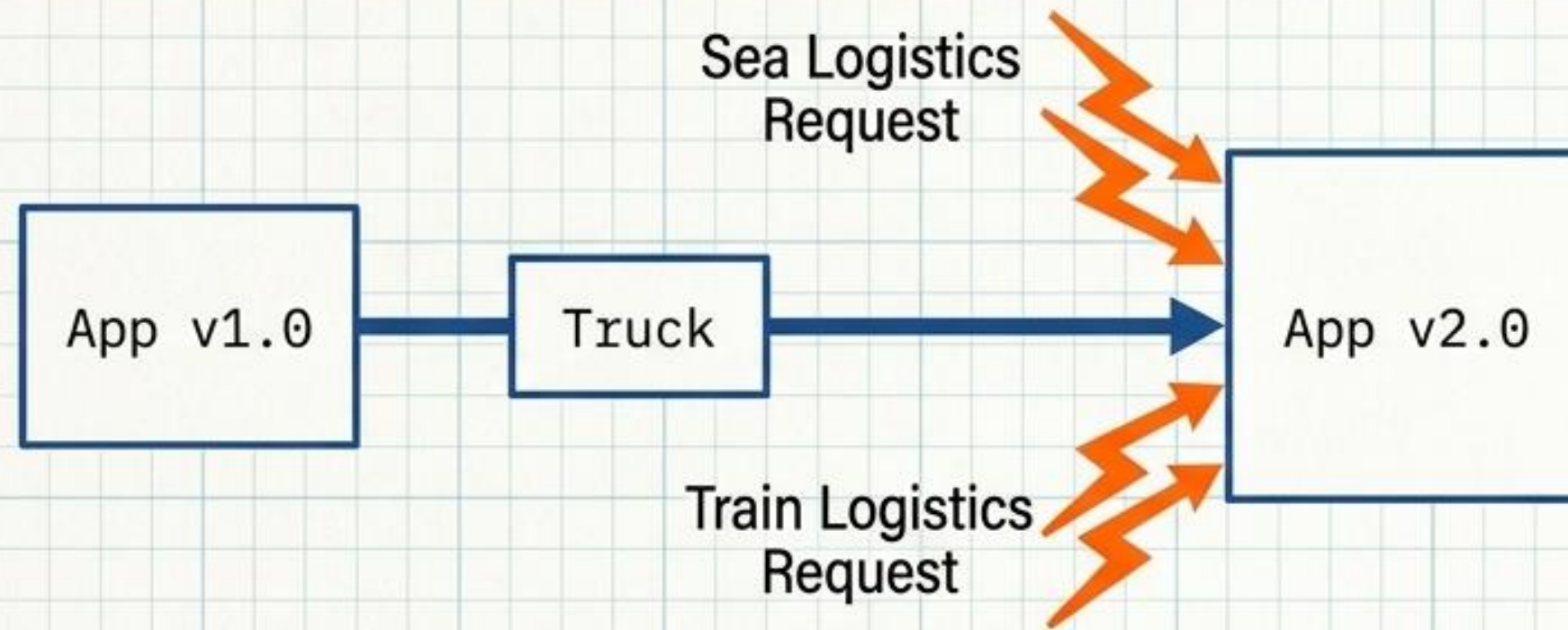
The Factory Method: Delegating Creation

The Factory Method (also known as a Virtual Constructor) provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.

Instructor's Note

Think of this as delegating the new keyword to a specialized method. Your main business logic doesn't need to know what exact object it's building—only how to use it once it's built.

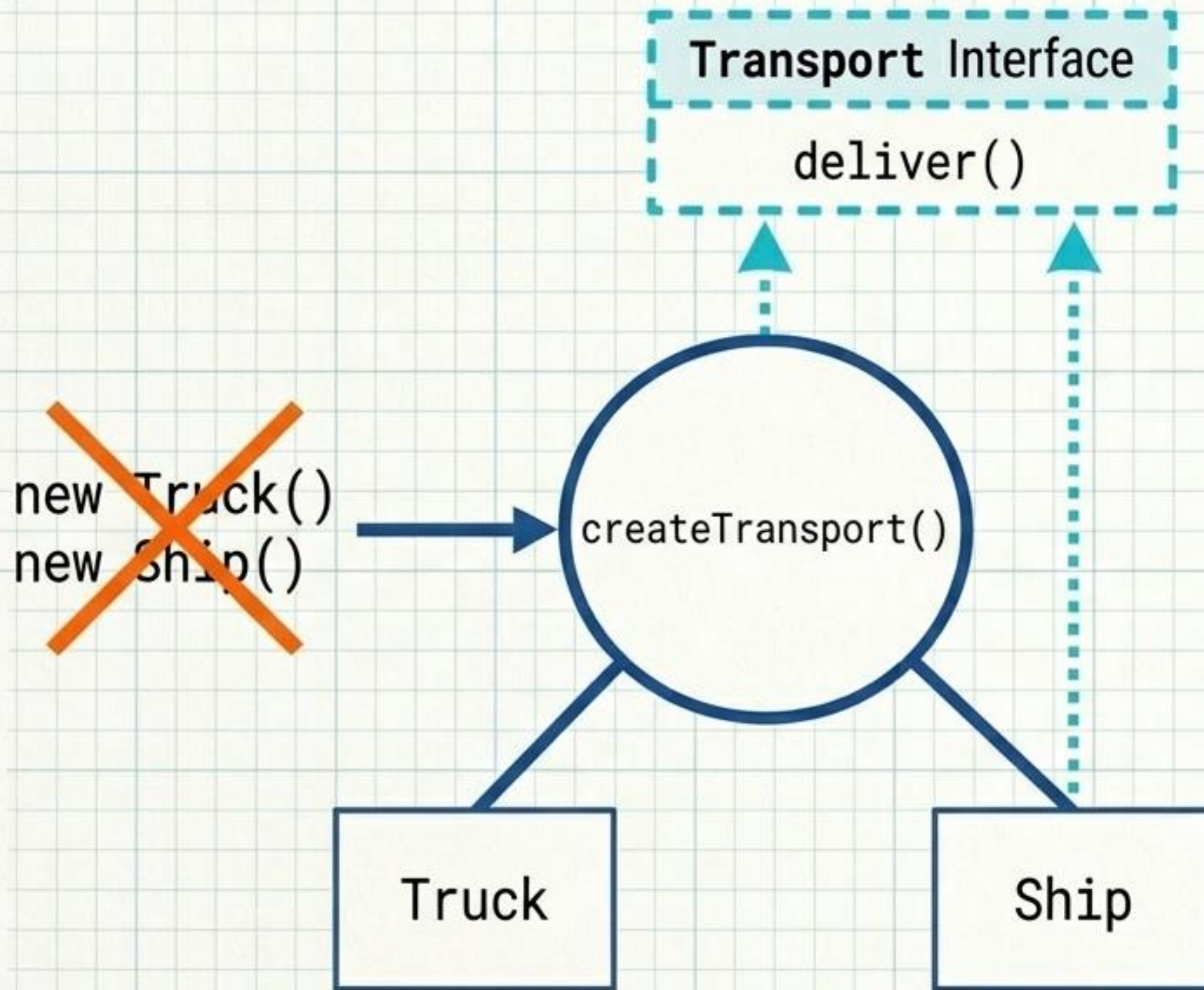
The Dilemma: Trapped by Tight Coupling



- **The Context:** Initial logistics app handles only road freight. Core logic is tightly coupled to the `Truck` class.
- **The Dilemma:** Adding a `Ship` class requires tearing open the entire codebase because the system is hardwired.
- **The Result:** Nasty, brittle code riddled with complex conditionals. Adding new types breaks the app all over again.

```
if (type == TRUCK) {  
    runTruckLogic();  
} else if (type == SHIP) {  
    runShipLogic();  
} else if (type == TRAIN) {  
    runTrainLogic();  
}  
// This codebase is a nightmare to maintain!
```


The Solution: Abstracting the Assembly Line

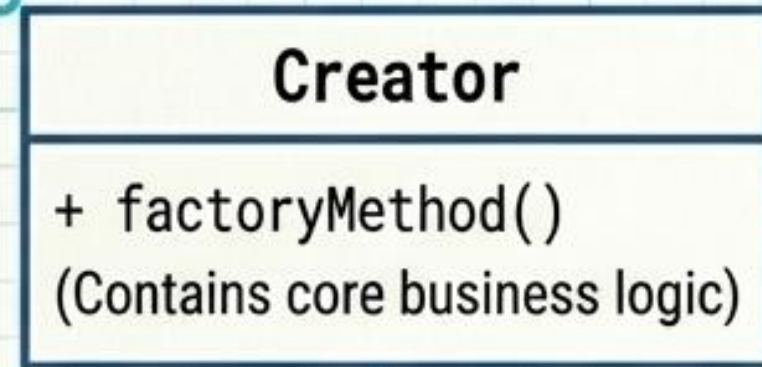


- **The Paradigm Shift:** Replace direct `new` object construction calls with calls to a special factory method. Returned objects are called *products*.
- **The Core Rule:** Subclasses can alter returned products **only** if they share a common base class or interface.
- **The Result:** Client code treats all products as an abstract `Transport`. It knows they have a `deliver()` method, making exact implementations irrelevant.

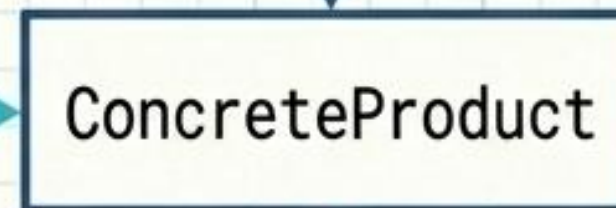
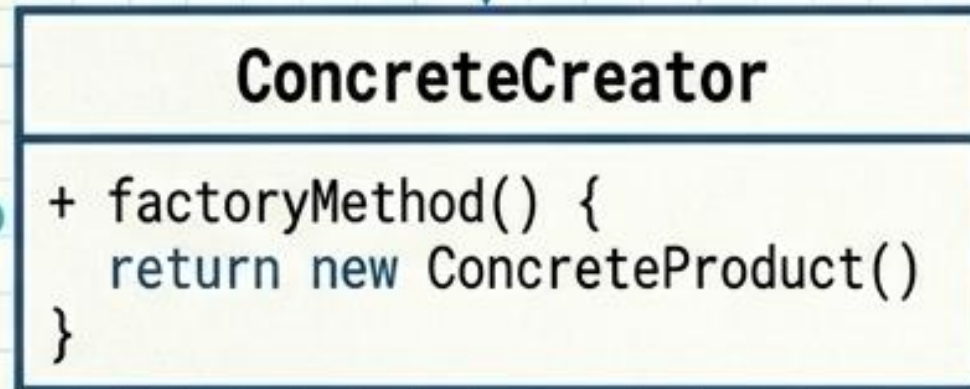
Anatomy of the Factory Method

Creator: Declares the factory method.

Primary responsibility is core business logic, decoupled from concrete classes.



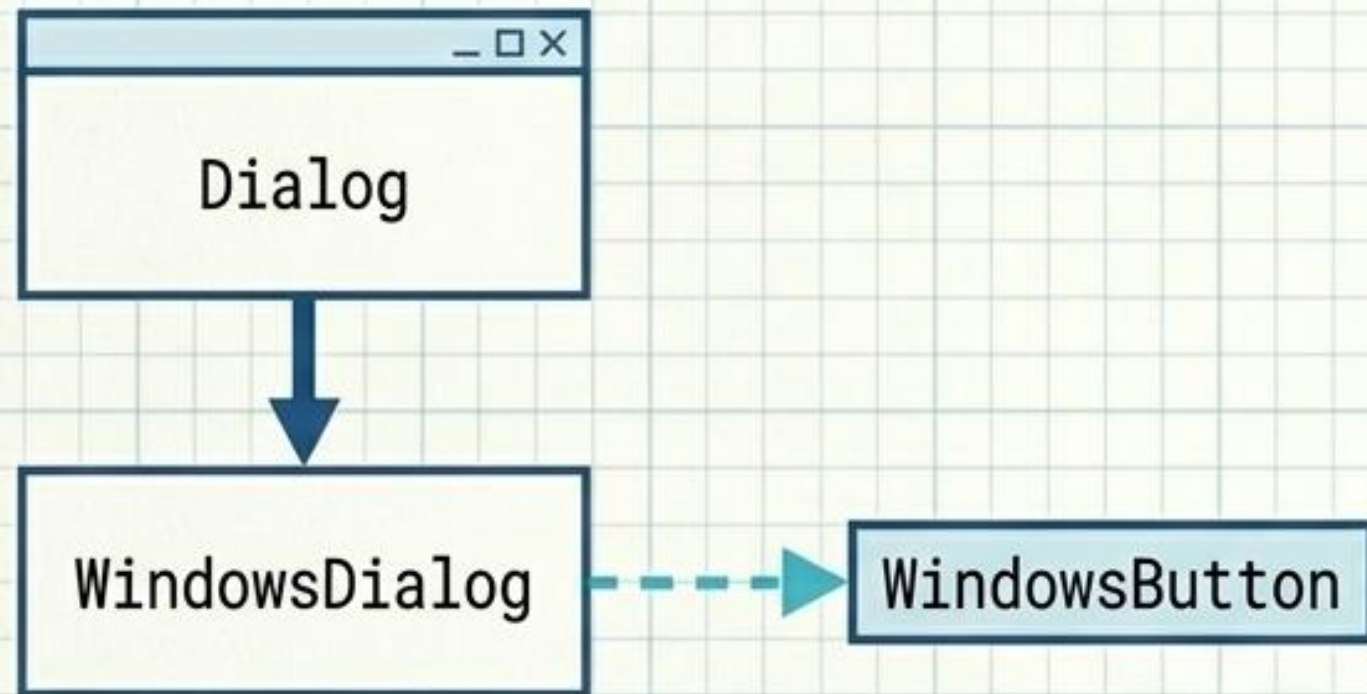
Product: Declares the common interface.



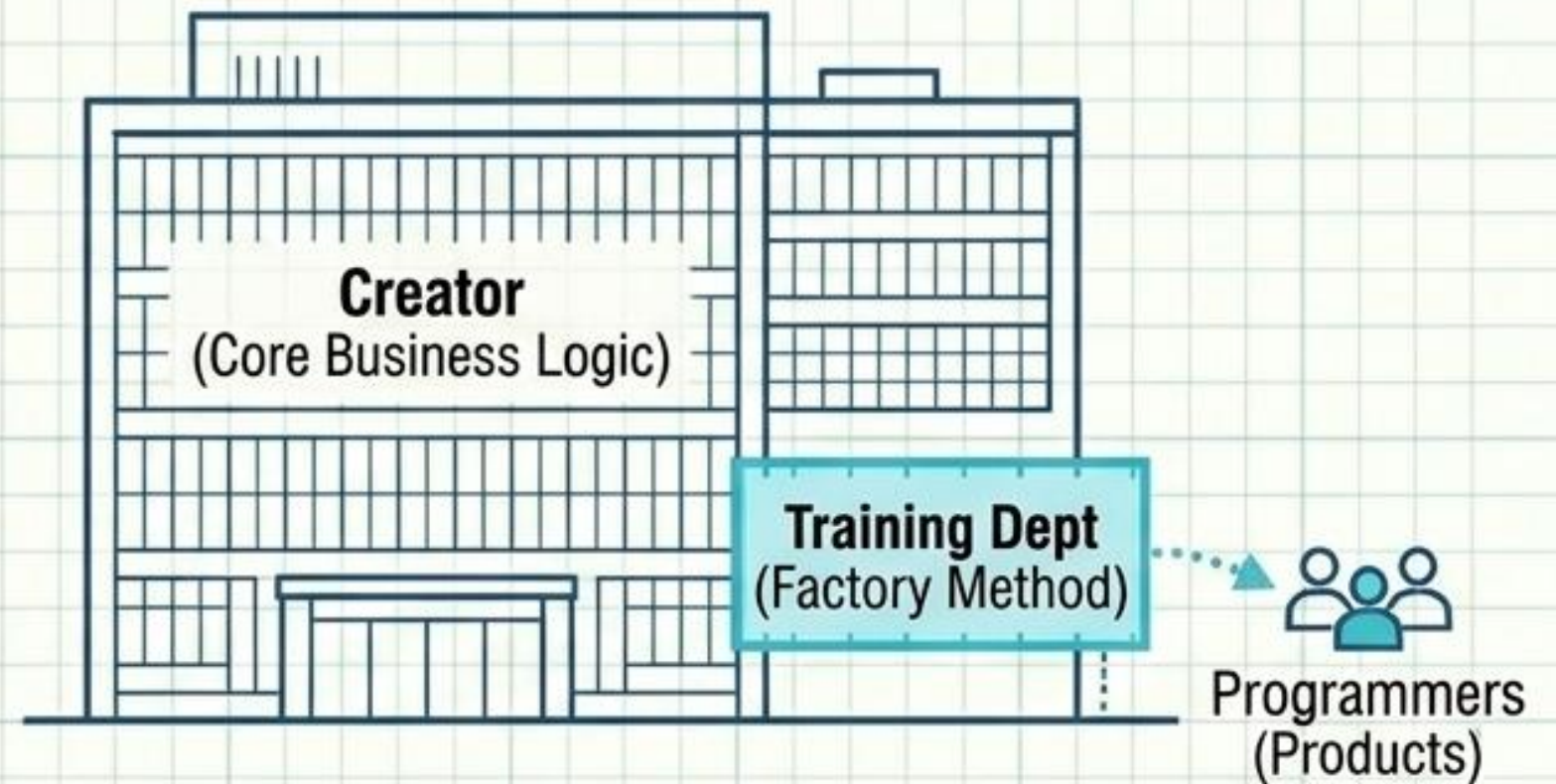
Concrete Creators: Override base factory method to return specific products.

Concrete Products: Distinct implementations of the interface.

Analogies: Cross-Platform UIs and Corporate Training



Software Example: Cross-platform UIs. A base Dialog class works strictly with abstract buttons. To port the app to Windows, create a WindowsDialog subclass that overrides the factory method to return a WindowsButton. Base logic remains untouched.



Real-World Analogy: A software company's primary function is writing code. To sustain this, an isolated training department produces programmers. The complex creation process is completely separated from the main business of coding.

Architectural Trade-offs: Factory Method



Advantages

Decoupling: Avoids tight coupling between the creator and the concrete products.

Single Responsibility Principle: Consolidates all product creation code into one easily supported location.

Open/Closed Principle: Safely introduce new types of products without breaking existing client code.



Disadvantages

Subclass Explosion: Code complexity can spike due to the requirement of introducing multiple new subclasses simply to implement the pattern.

Implementation Strategy: Best applied when introducing the pattern into an *existing* hierarchy of creator classes, rather than building from scratch.

When and How to Implement

```
abstract class Creator {  
    // The Factory Method  
    abstract Product createProduct();  
  
    // Core Business Logic  
    void someOperation() {  
        Product p = this.createProduct();  
        p.doStuff();  
    }  
}
```

When to Use

- When exact types and dependencies of objects are unknown beforehand.
- To provide framework users a way to extend internal components (e.g., returning a `RoundButton` instead of a default button).
- To save system resources by reusing heavy objects (like database connection pools) instead of rebuilding.

Implementation Logic

1. Ensure all products follow a shared interface.
2. Add an empty factory method returning that interface.
3. Replace `new` constructor calls with factory method calls.
4. Create subclasses to override the factory method.

Any questions?

The Macro View: Enter the Template Method

Behavioral Pattern

The Template Method (Algorithm Skeleton)



The Transition: Many architectures begin with a Factory Method. In fact, a Factory Method often serves as just one specialized step within a much larger architectural structure: **The Template Method**.

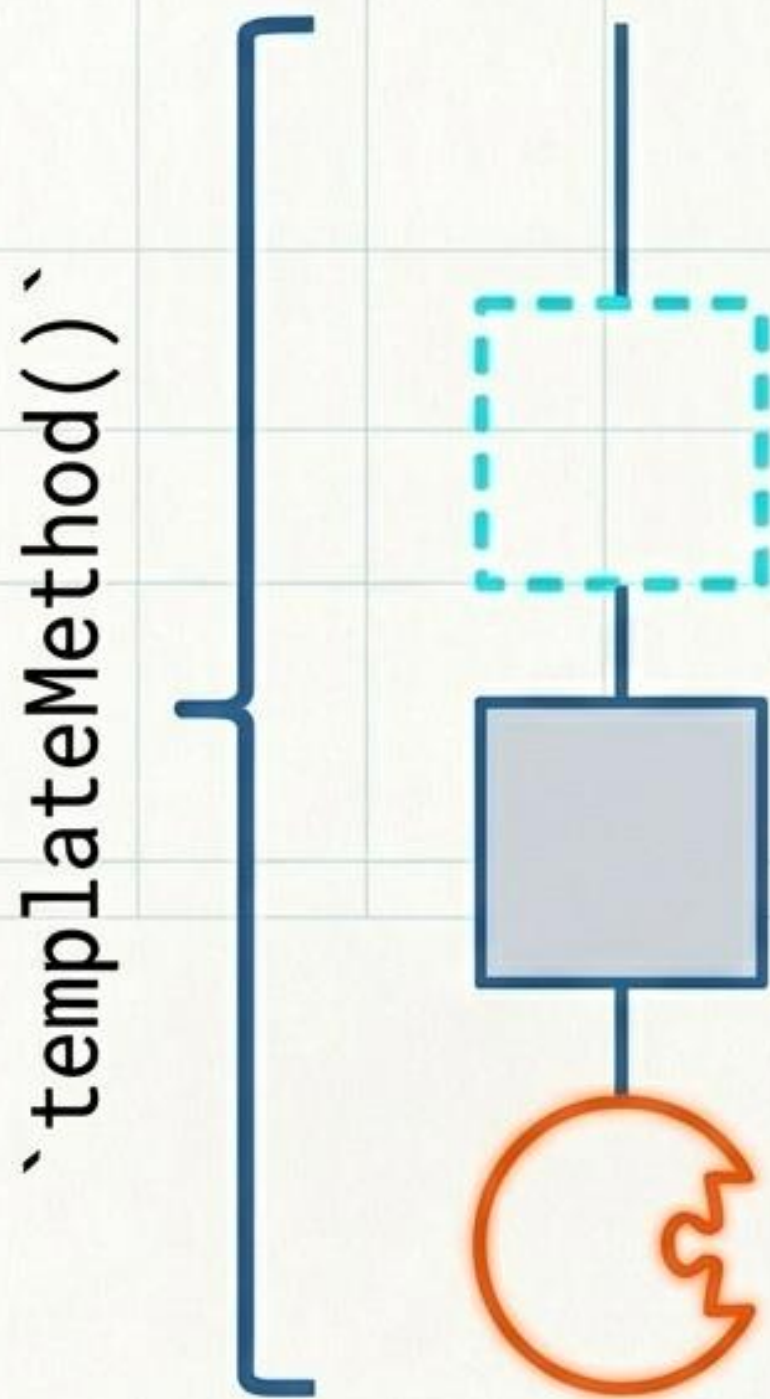
Intent: While Factory delegates *creation*, the Template Method defines the rigid skeleton of an entire *algorithm* in the superclass, letting subclasses override specific internal steps without ever changing the algorithm's overarching structure.

The Dilemma: Redundant Algorithms



- **The Context:** Building a data mining app that extracts and normalizes corporate data from DOC, CSV, and PDF formats.
- **The Dilemma:** Extraction requires unique logic, but the subsequent steps—analyzing raw data and composing reports—are exactly the same across all three classes.
- **The Result:** Widespread duplication. Modifying analysis logic requires “shotgun surgery” across multiple files, and client code is choked with conditionals.

The Solution: The Algorithmic Skeleton



The Paradigm Shift: Break the monolithic algorithm into steps. Centralize the sequence inside a single, strict *template method*.

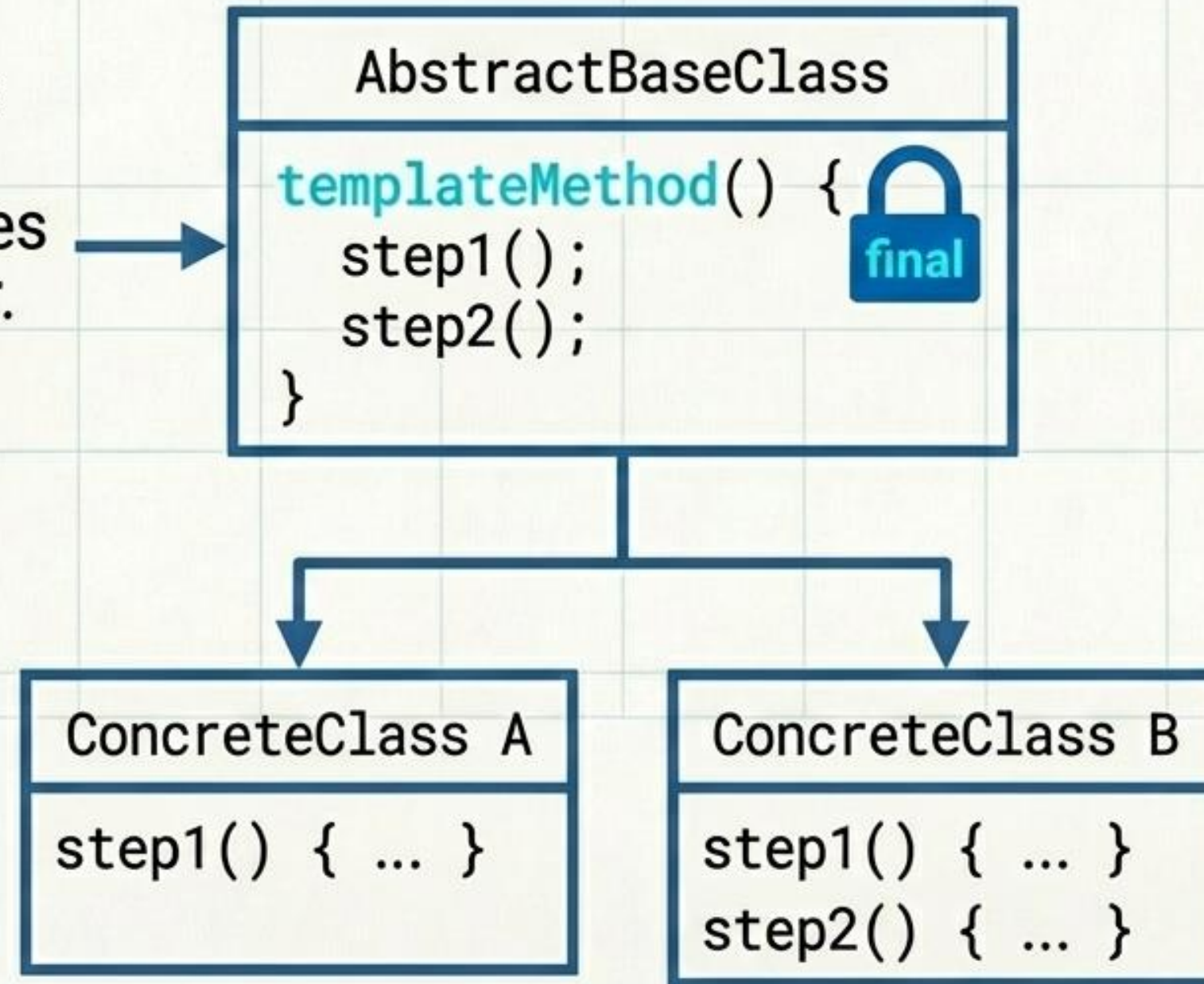
Abstract Steps: Have no implementation. Subclasses *must* implement them (e.g., parsing a specific file type).

Optional Steps: Have a default implementation in the base class. Subclasses *can* override them if needed.

Hooks: Optional steps with an empty body, placed before/after crucial points to provide subclasses a safe, isolated extension point.

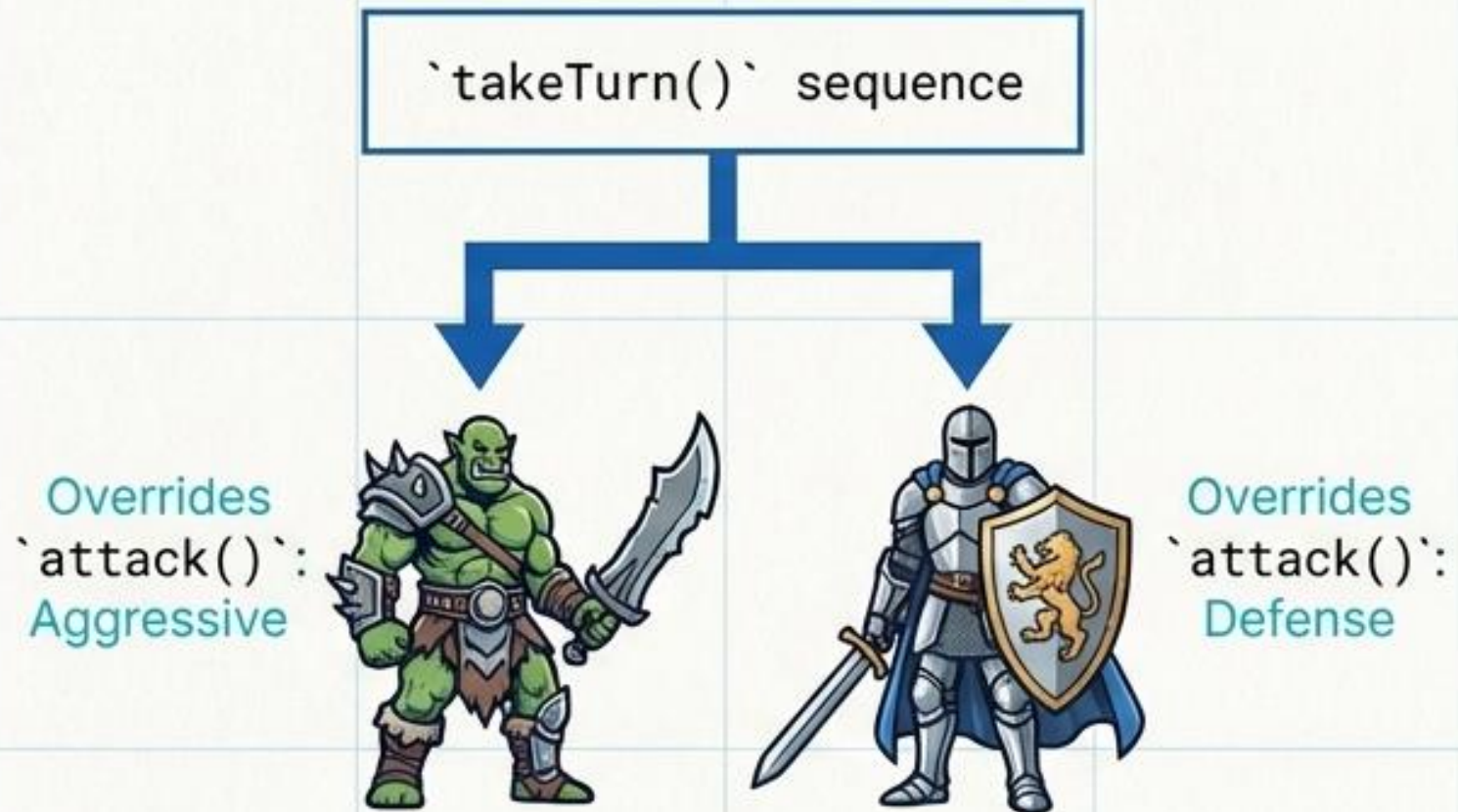
Anatomy of the Template Method

Abstract Base Class:
Houses the template method which dictates strict execution order. Marked `final` to mathematically prevent hijacking the sequence.



Concrete Classes:
Inherit the skeleton. Provide specific implementations for abstract steps. They **cannot* alter the overarching template method.

Analogies: Game AI and Mass Housing



Software Example: Strategy AI. A base `'AI'` class defines a strict turn sequence (gather, build, attack). The Orc subclass overrides `'attack()'` to be aggressive; Human focuses on defense. The turn sequence remains untouched.



Real-World Analogy: Mass housing. The architectural template dictates the foundation and framing sequence. Subclasses (homeowners) utilize hooks to alter specific details (paint, windows) making the house unique without compromising structural integrity.

Architectural Trade-offs: Template Method



Advantages

1. **Isolation:** Clients can override only specific parts of a massive algorithm, shielding them from upstream changes in other steps.

2. **DRY Principle:** Pulls identical code (like data analysis logic) up into a superclass, completely eliminating duplication.



Disadvantages

1. **Rigidity:** Clients are strictly limited by the skeleton provided by the superclass.

2. **Liskov Substitution Principle (LSP):** High risk of violating LSP if a subclass improperly suppresses or alters the intent of a default step implementation.

3. **Maintenance Burden:** Template methods become increasingly difficult to maintain as the number of sequential steps grows.

When and How to Implement

```
abstract class DataMiner {  
    // The Template Method  
    final void mine(String path) {  
        openFile(path);  
        extractData();  
        analyzeData();  
        closeHook();  
    }  
  
    // Abstract, Default, and Hook steps  
    abstract void extractData();  
    void analyzeData() { /* default */ }  
    void closeHook() { /* empty */ }  
}
```

When to Use

- To let clients extend specific steps of an algorithm without breaking the core structure.
- To consolidate multiple classes containing almost identical algorithms, preventing widespread 'shotgun surgery' during updates.

Implementation Logic

1. Break target algorithm into sequential steps.
2. Create abstract base class with a `final` template method.
3. Outline the execution sequence within the method.
4. Declare step methods as `abstract` or provide defaults.
5. Add empty `hook()` methods between crucial steps.